

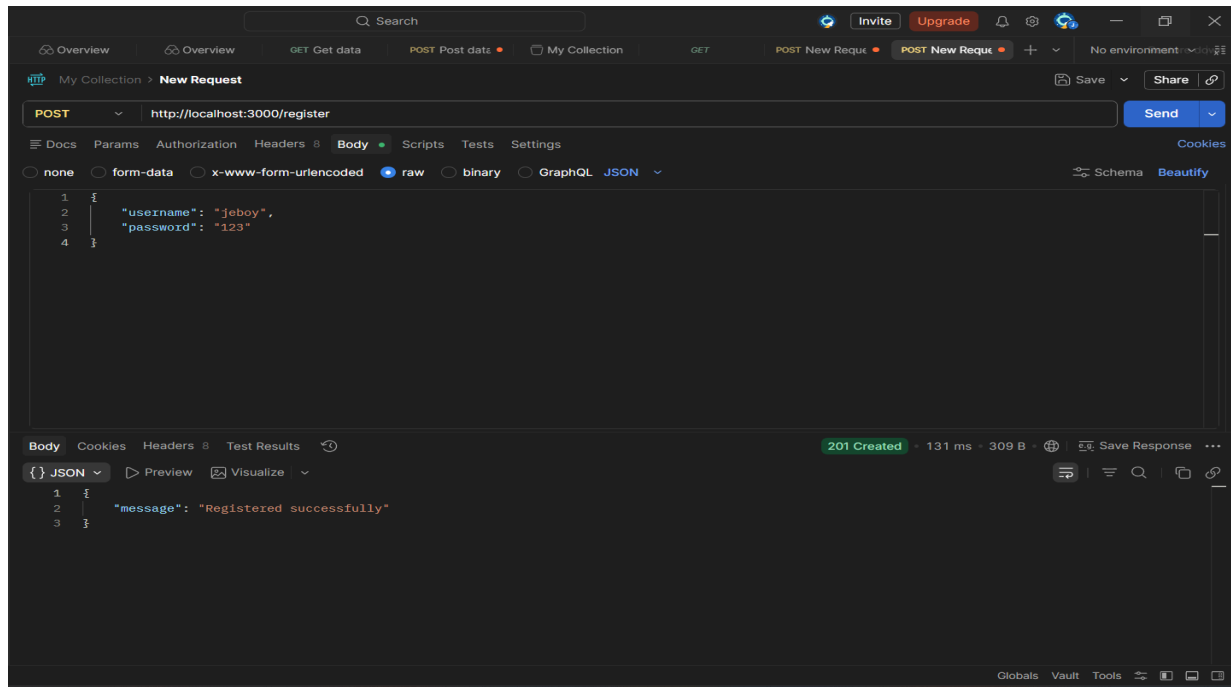
ALANGUILAN, JEBOY B.

BSCPE 2B

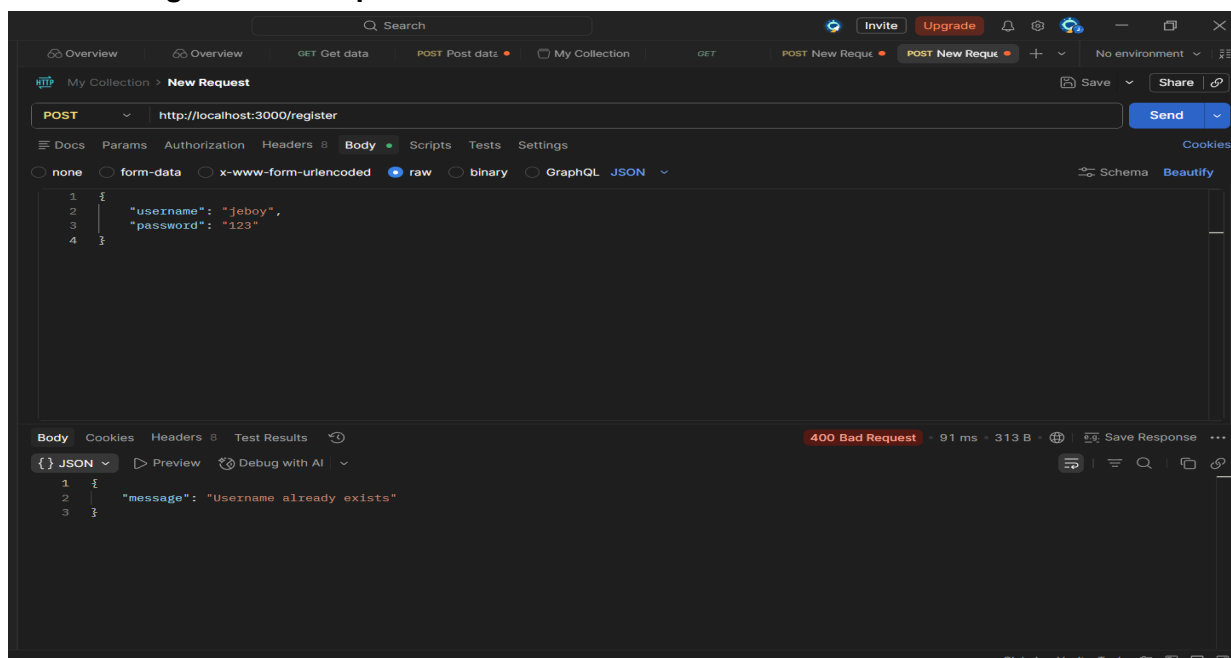
Quiz 1: Securing the API with JWT Authentication

A. Registration Tests (POST /register)

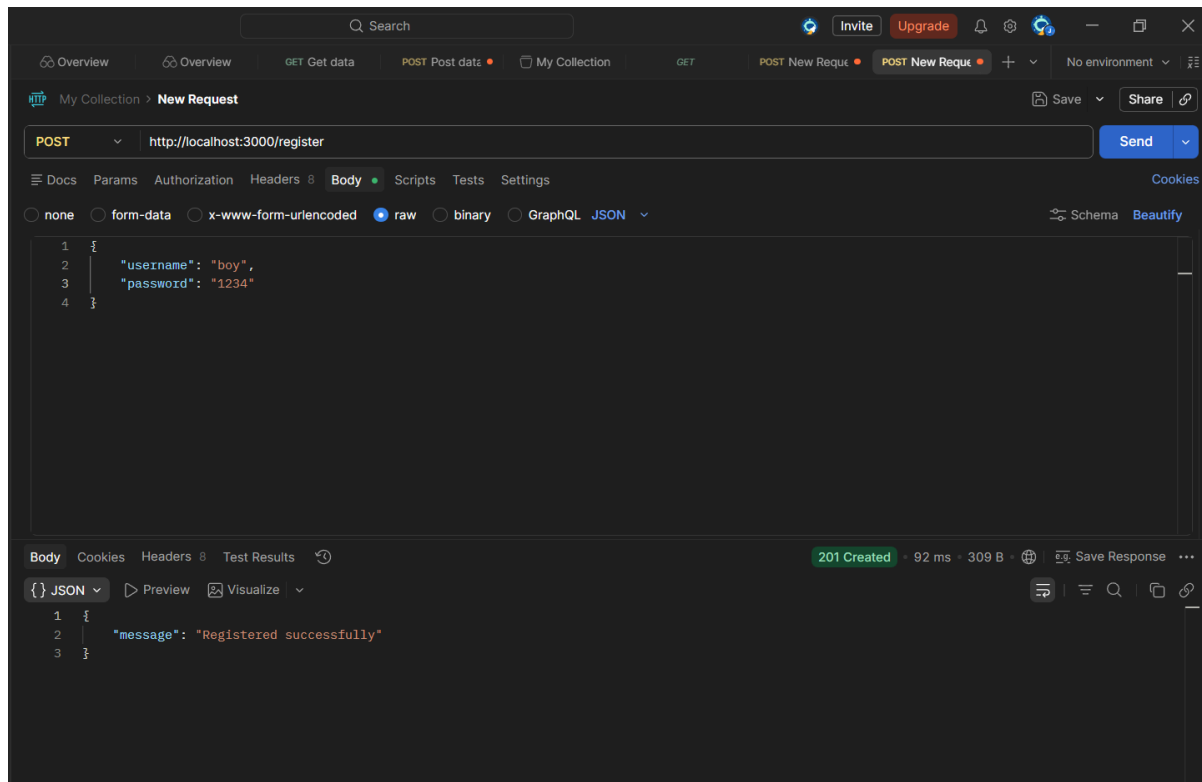
CASE 1: Register a new account



CASE 2: Register with duplicate username

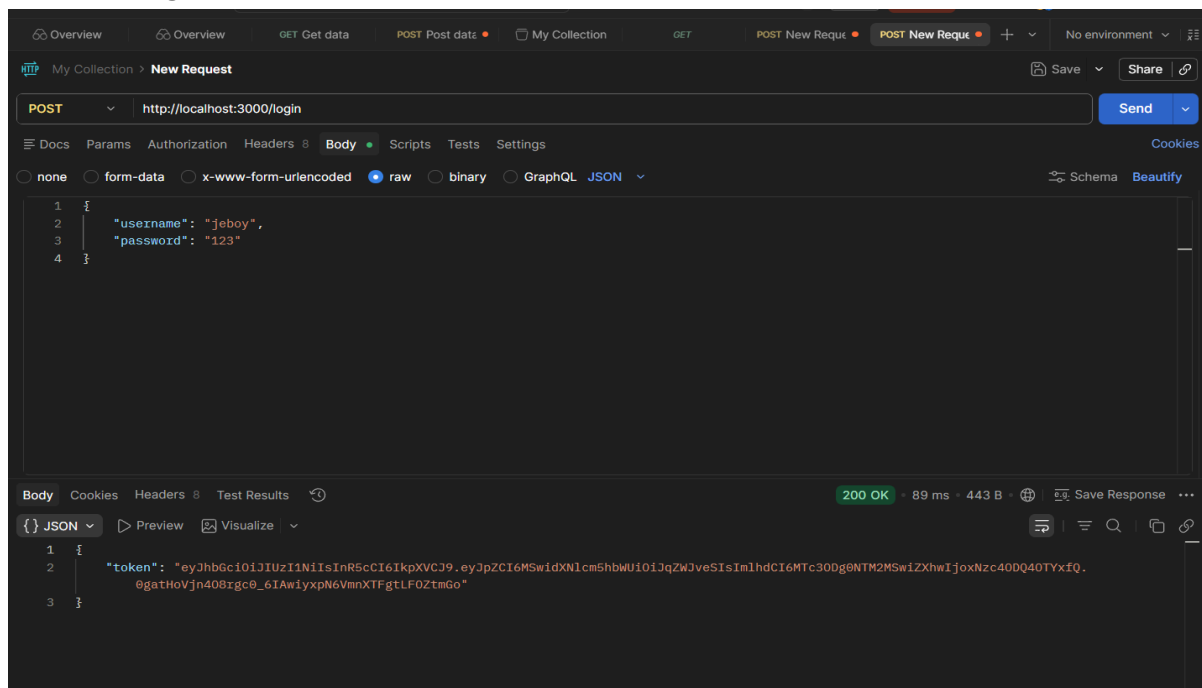


CASE 3: Register a second account



B. Login Tests (POST /login)

CASE 4: Login with correct credentials



CASE 5: Login with wrong password

The screenshot shows the Postman interface for a new request. The request is a POST to `http://localhost:3000/login`. The body is set to raw JSON with the following content:

```
1 {  
2   "username": "jeboy",  
3   "password": "1234"  
4 }
```

The response is displayed below the request, showing a status of **401 Unauthorized** with a response time of 90 ms and a body size of 310 B. The response body is JSON:

```
1 {  
2   "message": "Invalid credentials"  
3 }
```

CASE 6: Login with non-existing username

The screenshot shows the Postman interface for a new request. The request is a POST to `http://localhost:3000/login`. The body is set to raw JSON with the following content:

```
1 {  
2   "username": "ghost",  
3   "password": "1234"  
4 }
```

The response is displayed below the request, showing a status of **401 Unauthorized** with a response time of 4 ms and a body size of 310 B. The response body is JSON:

```
1 {  
2   "message": "Invalid credentials"  
3 }
```

CASE 7: Login with empty password

The screenshot shows the Postman interface for a POST request to `http://localhost:3000/login`. The request body is a JSON object with `username: "jeboy"` and an empty `password`. The response status is **401 Unauthorized** with a message of `"Invalid credentials"`.

```
POST http://localhost:3000/login
```

```
{  "username": "jeboy",  "password": ""}
```

401 Unauthorized · 82 ms · 310 B

```
{  "message": "Invalid credentials"}
```

C. Protected Route — GET /users

CASE 8: Access with valid token

The screenshot shows the Postman interface for a GET request to `http://localhost:3000/users`. The Authorization header is set to **Bearer Token** with the token `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`. The response status is **200 OK** with a JSON array of two user objects.

```
GET http://localhost:3000/users
```

Auth Type: Bearer Token

Token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

200 OK · 19 ms · 511 B

```
[  {    "id": 1,    "username": "jeboy",    "password": "$2b$10$fiXICgiNIzm09WPjZiFLkumQRAL51vbK2Qea7bdB9vmvQX7sXKYMC",    "status": "inactive"  },  {    "id": 3,    "username": "boy",    "password": "$2b$10$93TaFV1o.x4p/HuS.Ycg9e2RUHGkaNzxm0hVj1n0YEf/u4igbp5dW",    "status": "inactive"  }]
```

CASE 9: Access with no token

The screenshot shows the Postman interface for a new request. The method is GET and the URL is http://localhost:3000/users. The Authorization tab is selected, showing Bearer Token authentication. The Token field is empty. The response status is 401 Unauthorized, 4 ms, 308 B. The response body is JSON: {"message": "No token provided"}.

Overview Overview GET Get data POST Post data My Collection GET POST New Reque GET New Reque + No environment

My Collection > New Request Save Share

GET http://localhost:3000/users Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token

Body Cookies Headers 8 Test Results 401 Unauthorized 4 ms 308 B Save Response

{ JSON Preview Pass the correct auth credentials

```
1 {
2   "message": "No token provided"
3 }
```

CASE 10: Access with wrong token

The screenshot shows the Postman interface for a new request. The method is GET and the URL is http://localhost:3000/users. The Authorization tab is selected, showing Bearer Token authentication. The Token field contains the text 'wrong_token'. The response status is 403 Forbidden, 4 ms, 312 B. The response body is JSON: {"message": "Invalid or expired token"}.

Overview Overview GET Get data POST Post data My Collection GET POST New Reque GET New Reque + No environment

My Collection > New Request Save Share

GET http://localhost:3000/users Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token: wrong_token

Body Cookies Headers 8 Test Results 403 Forbidden 4 ms 312 B Save Response

{ JSON Preview Try with different auth credentials

```
1 {
2   "message": "Invalid or expired token"
3 }
```

CASE 11: Access with malformed header

The screenshot shows the Postman interface for a GET request to `http://localhost:3000/users`. The request has an `Authorization` header with a value `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXN...`. The response is `401 Unauthorized` with a status bar indicating `5 ms · 308 B`. The response body is JSON: `{ "message": "No token provided" }`.

Key	Value	Description
Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXN...	

```
{
  "message": "No token provided"
}
```

D. Protected Route — POST /add-user

CASE 12: Add user with valid token

The screenshot shows the Postman interface for a POST request to `http://localhost:3000/add-user`. The request is configured with `Auth Type` set to `Bearer Token` and a `Token` value `aVrzZ1vAwfYRGx2pnn_7EXzfbykE7tF3l`. The response is `200 OK` with a status bar indicating `21 ms · 269 B`. The response body is HTML: `User added`.

Auth Type: Bearer Token

Token: aVrzZ1vAwfYRGx2pnn_7EXzfbykE7tF3l

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

```
User added
```

CASE 13: Add user with no token

The screenshot shows the Postman interface for a POST request to `http://localhost:3000/add-user`. The **Authorization** tab is selected, and the **Auth Type** is set to **No Auth**. The **Body** tab is also selected, showing a JSON response:

```
{
  "message": "No token provided"
}
```

The status bar at the bottom indicates a **401 Unauthorized** response with a 5 ms duration and 308 B of data. The **Test Results** tab shows 0/1 tests passed.

CASE 14: Add user with wrong token

The screenshot shows the Postman interface for a POST request to `http://localhost:3000/add-user`. The **Authorization** tab is selected, and the **Auth Type** is set to **Bearer Token**. The **Token** field contains the value `invalidtoken`. The **Body** tab is also selected, showing a JSON response:

```
{
  "message": "Invalid or expired token"
}
```

The status bar at the bottom indicates a **403 Forbidden** response with a 4 ms duration and 312 B of data. The **Test Results** tab shows 0/1 tests passed.

E. Token Behavior Tests

CASE 15: Token from one login works for all protected routes

The screenshot shows the Postman interface with a GET request to `http://localhost:3000/users`. The request is configured with a Bearer Token: `U8YQIwTmkdMDfZIJ5GFxnCj8hSYXUk`. The response status is `200 OK` with a response time of `6 ms` and a body size of `446 B`. The response body is a JSON array of two user objects:

```
[{"id": 1, "username": "jeboy", "password": "$2b$10$wri/ewiwDLt6K4PurakSJ0yZVrFpAYJ0.aFui3UM5aBhii4jE01a.", "status": "active"}, {"id": 3, "username": "jeboyboy", "password": "", "status": ""}]
```

CASE 16: Different users get different tokens

The screenshot shows a text editor with the following content:

Tokens should be different strings

user A:
"token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJqZWJveSIsImIhdCI6MTc3ODg1MzE0MCwiZXhwIjoxNzc4ODU2NzQwfQ.6VfNfobmPn3K0LgOTaUlhFY0zwHm25sgdbQ_6JuaxTs"

user B:
"token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NCwidXNlcm5hbWUiOiJib3kiLCJpYXQiOiJlbnNzZg4NTMyNjYsImV4cCI6MTc3ODg1Njg2Nn0.9altPVu6aqK_HQXzC6FkEcqmpf4OS8mApaP3ESleROs"

CASE 17: Token contains user info

> JSON Web Token (JWT)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXN1cm5hbWUiOiJqZWJveSIsIm1hdCI6MTc3ODg1MzE0MCwiZXhwIjoxNzc4ODU2NzQwfQ.6VfNfobmPn3K0Lg0TaU1hfY0zwHm25sgdbQ_6JuaxTs|
```

JSON Claims Breakdown

```
{  "alg": "HS256",  "typ": "JWT"}
```

Decoded Payload

JSON Claims Breakdown

```
{  "id": 1,  "username": "jeboy",  "iat": 1778853140,  "exp": 1778856740}
```

CASE

18: Password is hashed in database

19: Registered accounts are saved

20: Added users are saved

Server: 127.0.0.1 » Database: fullstack_db » Table: users

Browse Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers

Showing rows 0 - 2 (3 total, Query took 0.0002 seconds.)

SELECT * FROM `users`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	username	password	status
☐ Edit Copy Delete	1	jeboy	\$2b\$10\$wri/eWrwDLt6K4PurakSJ0yZVrFpAYJ0.aFui3UM5aB...	active
☐ Edit Copy Delete	3	jeboyboy		
☐ Edit Copy Delete	4	boy	\$2b\$10\$R9FjZqGoeQI4QKMUtMybb.XwHbq9k.y/orvqdd11D...	active

↑ ☐ Check all With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query

Label: ☐ Let every user access this bookmark

Bookmark this SQL query